

SQL Notes: Execution Order, WHERE vs HAVING, Aggregate Functions, and Arithmetic Operations

1. SQL Logical Execution Order

Although we write SQL in this order:

```
SELECT  
FROM  
WHERE  
GROUP BY  
HAVING  
ORDER BY;
```

The database executes it in this order:

```
1. FROM  
2. WHERE  
3. GROUP BY  
4. Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)  
5. HAVING  
6. SELECT  
7. DISTINCT  
8. ORDER BY  
9. LIMIT / TOP
```

2. FROM

Purpose

- Selects the table(s).
- Performs JOINS.

Example

```
SELECT *  
FROM employee;
```

3. WHERE

Purpose

Filters **individual rows** before grouping.

Works Before

- GROUP BY
- Aggregate Functions

Can Use

- Column names
- Arithmetic on column values
- Comparison operators
- Logical operators

Example

```
SELECT *  
FROM employee  
WHERE salary > 50000;
```

Example with arithmetic

```
SELECT *  
FROM employee  
WHERE salary - bonus > 50000;
```

Reason:

Each row already contains `salary` and `bonus`.

WHERE Cannot Use

- COUNT()
- SUM()
- AVG()
- MIN()
- MAX()
- Aliases created in SELECT

Wrong

```
SELECT department, COUNT(*)  
FROM employee  
WHERE COUNT(*) > 5  
GROUP BY department;
```

Reason:

When SQL executes WHERE, COUNT() has not been calculated.

4. GROUP BY

Purpose

Groups rows having the same values.

Example

```
SELECT department, COUNT(*)  
FROM employee  
GROUP BY department;
```

5. Aggregate Functions

Aggregate functions work on multiple rows and return a single value.

They are calculated **after GROUP BY**.

Common aggregate functions

```
COUNT()  
SUM()  
AVG()  
MIN()  
MAX()
```

Example

```
SELECT
COUNT(*),
SUM(salary),
AVG(salary),
MIN(salary),
MAX(salary)
FROM employee;
```

6. HAVING

Purpose

Filters grouped (aggregated) data.

Runs after aggregate functions.

Example

```
SELECT department,
COUNT(*) AS total
FROM employee
GROUP BY department
HAVING COUNT(*) > 5;
```

Another example

```
SELECT department,
SUM(salary) AS total_salary
FROM employee
GROUP BY department
HAVING SUM(salary) > 100000;
```

7. SELECT

Purpose

Displays columns and expressions.

Creates aliases.

Performs calculations.

Example

```
SELECT
  salary,
  bonus,
  salary - bonus AS net_salary
FROM employee;
```

Aggregate arithmetic

```
SELECT
  COUNT(*) - COUNT(DISTINCT city) AS difference
FROM station;
```

8. DISTINCT

Purpose

Removes duplicate rows from the final result.

Example

```
SELECT DISTINCT city
FROM station;
```

Count unique values

```
SELECT COUNT(DISTINCT city)
FROM station;
```

9. ORDER BY

Purpose

Sorts the final output.

Example

```
SELECT *  
FROM employee  
ORDER BY salary DESC;
```

Can also use aggregate functions.

```
SELECT department,  
COUNT(*) AS total  
FROM employee  
GROUP BY department  
ORDER BY total DESC;
```

10. LIMIT

Returns only the required number of rows.

Example

```
SELECT *  
FROM employee  
LIMIT 5;
```

WHERE vs HAVING

WHERE	HAVING
Filters rows	Filters groups
Before GROUP BY	After GROUP BY

WHERE	HAVING
Cannot use aggregate functions	Can use aggregate functions
Works on row data	Works on grouped data

Aggregate Functions Usage

Clause	COUNT	SUM	AVG	MIN	MAX
SELECT	✓	✓	✓	✓	✓
WHERE	✗	✗	✗	✗	✗
HAVING	✓	✓	✓	✓	✓
ORDER BY	✓	✓	✓	✓	✓

Why Aggregate Functions Cannot Be Used in WHERE

Execution order

```

FROM
↓
WHERE
↓
GROUP BY
↓
COUNT(), SUM(), AVG(), MIN(), MAX()
↓
HAVING
↓
SELECT

```

When SQL reaches WHERE:

- COUNT() does not exist yet.
- SUM() does not exist yet.
- AVG() does not exist yet.
- MIN() does not exist yet.
- MAX() does not exist yet.

Therefore, they cannot be used in WHERE.

Arithmetic Operations

Arithmetic on Row Values

Allowed in SELECT

```
SELECT salary - bonus  
FROM employee;
```

Allowed in WHERE

```
SELECT *  
FROM employee  
WHERE salary + bonus > 50000;
```

Reason:

Both values belong to each row.

Arithmetic on Aggregate Values

Allowed in SELECT

```
SELECT  
COUNT(*) - COUNT(DISTINCT city) AS difference  
FROM station;
```

Allowed in HAVING

```
SELECT department,  
SUM(salary)  
FROM employee  
GROUP BY department  
HAVING SUM(salary) > 100000;
```

Reason:

Aggregate values are already calculated.

Aliases

Alias means giving a temporary name.

Example

```
SELECT salary AS employee_salary
FROM employee;
```

Example

```
SELECT COUNT(*) AS total
FROM employee;
```

Aliases are available in:

- SELECT output
- ORDER BY (in most databases)

Aliases are **not** available in WHERE because WHERE executes before SELECT.

Wrong

```
SELECT salary AS s
FROM employee
WHERE s > 50000;
```

Correct

```
SELECT salary AS s
FROM employee
WHERE salary > 50000;
```

Your STATION Question

Wrong

```
SELECT COUNT(city) AS total_city,  
DISTINCT city AS unique_city  
FROM station  
WHERE total_city - unique_city;
```

Problems

1.

```
DISTINCT city
```

Wrong.

Correct

```
COUNT(DISTINCT city)
```

2.

```
WHERE total_city - unique_city;
```

Wrong because

- WHERE filters rows.
- total_city is an alias.
- unique_city is an alias.
- Aliases are created in SELECT.
- WHERE executes before SELECT.
- Aggregate values do not exist yet.

Correct

```
SELECT  
COUNT(city) - COUNT(DISTINCT city) AS difference  
FROM station;
```

Can Aggregate Functions Be Used in WHERE?

COUNT()

✗ Wrong

```
SELECT *  
FROM employee  
WHERE COUNT(*) > 5;
```

✓ Correct

```
SELECT department  
FROM employee  
GROUP BY department  
HAVING COUNT(*) > 5;
```

SUM()

✗ Wrong

```
SELECT *  
FROM employee  
WHERE SUM(salary) > 100000;
```

✓ Correct

```
SELECT department  
FROM employee  
GROUP BY department  
HAVING SUM(salary) > 100000;
```

AVG()

✗ Wrong

```
SELECT *  
FROM employee  
WHERE AVG(salary) > 50000;
```

✓ Correct

```
SELECT department  
FROM employee  
GROUP BY department  
HAVING AVG(salary) > 50000;
```

MIN()

✗ Wrong

```
SELECT *  
FROM employee  
WHERE MIN(salary) > 20000;
```

✓ Correct

```
SELECT department  
FROM employee  
GROUP BY department  
HAVING MIN(salary) > 20000;
```

MAX()

✗ Wrong

```
SELECT *  
FROM employee  
WHERE MAX(salary) > 100000;
```

✓ Correct

```
SELECT department
FROM employee
GROUP BY department
HAVING MAX(salary) > 100000;
```

Important Interview Rules

Rule 1

WHERE filters **rows**.

Rule 2

HAVING filters **groups**.

Rule 3

Aggregate functions cannot be used in WHERE.

Rule 4

Aggregate functions can be used in SELECT, HAVING, and ORDER BY.

Rule 5

Row-level arithmetic can be used in SELECT and WHERE.

Rule 6

Aggregate arithmetic can be used in SELECT and HAVING.

Rule 7

Aliases created in SELECT cannot be used in WHERE.

Rule 8

`COUNT(DISTINCT column)` counts only unique values.

Rule 9

Always remember the execution order:

```
FROM
↓
WHERE
↓
GROUP BY
↓
Aggregate Functions
↓
HAVING
↓
SELECT
↓
DISTINCT
↓
ORDER BY
↓
LIMIT
```

One-Line Memory Trick

- **FROM** → Get the data.
- **WHERE** → Filter rows.
- **GROUP BY** → Make groups.
- **Aggregate Functions** → Calculate values.
- **HAVING** → Filter groups.
- **SELECT** → Display results and calculations.
- **DISTINCT** → Remove duplicate result rows.
- **ORDER BY** → Sort the output.
- **LIMIT** → Return the required number of rows.

Golden Rule

If your condition depends on individual rows, use `WHERE`. If your condition depends on `COUNT()`, `SUM()`, `AVG()`, `MIN()`, or `MAX()`, use `HAVING`.